

NanoBench: A Multi-Task Benchmark Dataset for Nano-Quadrotor System Identification, Control, and State Estimation

Syed Izzat Ullah, José Baca

Department of Engineering

Texas A&M University-Corpus Christi, Corpus Christi, TX, USA

{syed.izzatullah, jose.baca}@tamucc.edu

Abstract—Existing aerial-robotics benchmarks target vehicles from hundreds of grams to several kilograms and typically expose only high-level state data. They omit the actuator-level signals required to study nano-scale quadrotors, where low-Reynolds-number aerodynamics, coreless DC motor nonlinearities, and severe computational constraints invalidate models and controllers developed for larger vehicles. We introduce *NanoBench*, an open-source multi-task benchmark collected on the commercially available Crazyflie 2.1 nano-quadrotor (takeoff weight 27 g) in a Vicon motion capture arena. The dataset comprises 172 flight recordings spanning multi-frequency excitation, geometric trajectory tracking at three speed regimes, and long-duration battery-drain hover. Each recording provides synchronized Vicon ground truth, raw IMU data, onboard extended Kalman filter (EKF) estimates, PID controller internals, and motor PWM commands at 100 Hz, alongside battery telemetry at 10 Hz, cross-aligned by gyroscope-based correlation on a 1 ms search grid. NanoBench defines standardized evaluation protocols, train/test splits, and open-source baselines for three tasks: nonlinear system identification, closed-loop controller benchmarking, and onboard state estimation assessment. To our knowledge, it is the first public dataset to jointly provide actuator commands, controller internals, and estimator outputs with millimeter-accurate ground truth on a commercially available nano-scale aerial platform.

Anonymous dataset and code:

<https://github.com/syediu/nanobench>

Video: https://syediu.github.io/assets/img/nanobench_demo.mp4

I. INTRODUCTION

Learning-based methods have substantially advanced quadrotor autonomy. Deep reinforcement learning policies match professional pilots in drone racing [1], adaptive controllers trained on minutes of flight data enable agile flight in strong wind [2], and model predictive controllers with learned residual dynamics outperform rigid-body baselines [3]. These techniques depend on instrumented datasets for training and reproducible evaluation. Most publicly available datasets that support such evaluation were collected on platforms weighing hundreds of grams to several kilograms. For nano-scale quadrotors, platforms below 50 g, that are widely used in embedded autonomy, safe learning, swarm robotics, autonomous navigation [4]–[9], no open benchmark exposes the full closed-loop stack.

Nano-scale quadrotors such as the Crazyflie 2.1 operate in aerodynamic and computational regimes that differ qualitatively from those of larger vehicles. Propellers with diameters

below 50 mm operate at Reynolds numbers on the order of 10^4 , where laminar separation bubbles and viscous losses significantly affect thrust and torque [10]. Coreless DC motors introduce deadbands and response lags absent in brushless propulsion [11], and proximity to surfaces induces strong ground-effect disturbances at low altitude [12]. Computationally, a 168 MHz Cortex-M4 microcontroller restricts onboard estimation to lightweight extended Kalman filters [13].

These physical and computational constraints motivate systematic characterization of platform-level uncertainty in nano-scale aerial systems, including actuation nonlinearities, estimator behavior, and battery-dependent performance [14], yet no standardized benchmark exists to study nano-scale dynamics under these constraints. Thrust models transfer poorly across Reynolds-number regimes [15], policies trained in simulation degrade on physical hardware because of inaccurate thrust modeling [16], [17], and lightweight estimators require precisely synchronized multi-modal recordings for proper validation [18]. Despite this demand, no existing dataset provides actuator commands, controller internals, and estimator outputs with external ground truth on a commercially available nano-scale platform. System identification, control, and state estimation methods for this class of vehicle are currently evaluated on private or synthetic data, which precludes systematic comparison [19].

This paper presents *NanoBench*, an open-source multi-task benchmark that addresses this gap. The dataset is collected on the standard Crazyflie 2.1 quadrotor inside a Vicon motion capture arena using a data collection framework built on direct radio communication through the cflib Python library. Our contributions are:

- 1) **Nano-quadrotor flight dataset.** We release 172 flight recordings on the Crazyflie 2.1 covering multi-frequency excitation, geometric trajectory tracking at three speed regimes, and long-duration battery-drain hover. Every recording includes time-synchronized Vicon ground truth, raw IMU measurements, onboard EKF state estimates, PID controller internals, and motor PWM commands at 100 Hz, with battery telemetry at 10 Hz.
- 2) **Cross-correlation time alignment.** We develop and validate a synchronization procedure that cross-correlates onboard gyroscope rates with Vicon-derived angular

velocity to estimate and correct the firmware-to-host clock offset. Across all flights, the residual misalignment is around 1 ms.

- 3) **Multi-task evaluation suite.** We define train/test splits, metrics, and reporting conventions for three tasks (system identification, controller benchmarking, and state estimation) and release open-source baseline implementations that establish reference performance on each.

The remainder of this paper is organized as follows. Section II positions NanoBench against prior work. Section III describes the platform, acquisition pipeline, synchronization procedure, trajectory design, and task formulations. Section IV reports dataset statistics and baseline results, Section IV-E provides the limitations, and Section V discusses the conclusions.

II. RELATED WORK

A. Aerial Robot Datasets and Benchmarks

Aerial robotics datasets can be partitioned by the downstream tasks they support and the platform scales they cover. The EuRoC MAV dataset [20], UZH-FPV [22], NTU VIRAL [24], and INSANE [25] provide high-fidelity exteroceptive sensor data for visual-inertial odometry and multi-sensor fusion. However, these benchmarks treat the vehicle as a passive sensor carrier and omit actuator and control signals. As a result, they cannot support closed-loop dynamics modeling, controller evaluation, or estimator benchmarking.

Actuation-aware datasets capture the inputs necessary for system identification. The Blackbird dataset [21] and Agilicious framework [26] log motor commands for agile flight research. However, they feature quadrotors weighing near 1 kg equipped with brushless motors. These heavy platforms exhibit rigid-body dynamics that differ fundamentally from the coreless DC actuation and aerodynamic nonlinearities of nano-quadrotors, precluding direct transfer of models or control policies.

At the nano scale, Busetto et al. [23] recently released a system identification dataset for a brushless Crazyflie variant that logs motor RPM. The dataset covers four trajectories and targets system identification only; it does not expose onboard estimator outputs, controller internals, or battery telemetry (Table I). Multi-task evaluation on the standard, brushed Crazyflie 2.1 therefore remains unaddressed.

B. System Identification for Aerial Vehicles

Quadrotor dynamics modeling ranges from calibrated physics to fully learned representations. Forster [11] established baseline quadratic thrust parameters via least-squares fitting. Torrente et al. [3] and NeuroBEM [15] augment nominal rigid-body equations with data-driven residuals to capture unmodeled aerodynamic effects. Neural ordinary differential equations offer purely data-driven alternatives. Zhou et al. [27] trained Neural ODEs for multi-step state prediction, while KNODE-MPC [28] and O’Connell et al. [2] embed adapted models within predictive controllers to improve tracking accuracy.

Existing simulation ecosystems configure these learned and analytical models with static coefficients [29]. This constant-parameter assumption ignores the substantial thrust degradation caused by single-cell LiPo discharge, and no existing benchmark provides the synchronized voltage and kinematic data required to model this non-stationary effect.

C. Control of Nano-Quadrotors

The Crazyflie is a primary testbed for control architectures, including cascaded PID, geometric controllers on $SE(3)$, and minimum-snap trackers [30]. Safe reinforcement learning frameworks predominantly use it [5], and recent work achieves zero-shot sim-to-real transfer [17]. Song et al. [16] showed that thrust calibration errors are the dominant source of tracking degradation during such transfer. Open-source ecosystems simplify swarm coordination and simulated execution [6], [7], [29], [31] and lower the barrier to multi-agent research.

Despite this active development, each research group runs its own private flight experiments and reports numbers against its own reference trajectories. Without a shared dataset, differences in results between papers reflect hardware setup as much as algorithm design.

D. State Estimation for Micro Aerial Vehicles

State estimation on sub-50 g platforms faces extreme hardware limitations. Visual-inertial odometry is too expensive for a 168 MHz Cortex-M4 [32], so nano-quadrotors run lightweight EKF with characterized IMU noise models [13], [18].

Evaluating these filters requires the estimator’s internal state recorded alongside an external reference at matched timestamps; without that pairing, sensor noise cannot be separated from filter divergence.

III. METHODOLOGY

A. Hardware Platform

We use the Crazyflie 2.1 nano-quadrotor (Bitcraze AB) as the data collection platform. The vehicle measures $92 \times 92 \times 29$ mm and weighs 27 g in the stock configuration with a 250 mA h single-cell LiPo installed. Four 7×16 mm coreless DC motors drive 46 mm four-blade propellers; at full charge (4.2 V), combined peak thrust is approximately 0.6 N. All NanoBench recordings were flown in an instrumented configuration that adds retroreflective markers and the wireless-charging deck, raising the measured flying mass to 40.85 g. The thrust-to-weight ratio is therefore ≈ 2.2 for the stock airframe but ≈ 1.5 as flown; we use the instrumented mass throughout. The onboard STM32F405 (168 MHz Cortex-M4F, 192 kB SRAM) runs a cascaded PID controller and an extended Kalman filter; a secondary nRF51822 handles radio communication. Inertial data are provided by a Bosch BMI088 six-axis IMU. Motor commands are 16-bit PWM values in $[0, 65535]$, and the unregulated single-cell battery directly couples state of charge to actuator performance.

Ground-truth 6-DoF pose is provided by a 12-camera Vicon system covering a $6 \times 4 \times 2$ m volume. Retroreflective markers

TABLE I: Comparison of aerial robotics datasets. NanoBench is the first to combine actuator-level data, onboard estimator outputs, and controller internals on a commercially available nano-scale platform within a multi-task evaluation framework.

Feature	EuRoC [20]	Blackbird [21]	UZH-FPV [22]	Busetto et al. [23]	NanoBench (Ours)
Platform mass	~2 kg	~1 kg	~0.8 kg	<50 g	~27 g
Platform availability	Discontinued	Custom	Custom	Commercial	Commercial
Number of trajectories	11	176	27	4	172
Ground truth system	Vicon/Leica	OptiTrack	Leica	MoCap	Vicon
Ground truth rate	200 Hz	360 Hz	200 Hz	Not specified	100 Hz
Raw IMU data	✓	✓	✓	–	✓
Motor commands	–	✓(RPM)	–	✓(RPM)	✓(PWM)
Onboard estimator output	–	–	–	–	✓
Controller internals	–	–	–	–	✓
Battery telemetry	–	–	–	–	✓
Camera data	Stereo	Synthetic	Stereo + event	–	–
Multi-task evaluation	–	–	–	SysID only	SysID + Control + Estimation
Calibration tools	–	–	–	–	✓
Open-source	✓	✓	Partial	✓	✓

TABLE II: Recorded signals and nominal sampling rates.

Signal	Source	Rate (Hz)
Position ($\mathbf{p} \in \mathbb{R}^3$), orientation ($\mathbf{q} \in S^3$)	Vicon	100
Linear velocity ($\dot{\mathbf{p}} \in \mathbb{R}^3$)	Vicon (derived)	100
Accelerometer ($\mathbf{a}_{\text{IMU}} \in \mathbb{R}^3$) [G]	BMI088	100
Gyroscope ($\boldsymbol{\omega}_{\text{IMU}} \in \mathbb{R}^3$) [rad/s]	BMI088	100
EKF state ($\hat{\mathbf{p}}, \hat{\dot{\mathbf{p}}}, \hat{\boldsymbol{\eta}}$)	Firmware	100
Controller setpoints ($\mathbf{p}_{\text{ref}}, \psi_{\text{ref}}$)	Firmware	100
PID outputs (roll, pitch, yaw, thrust)	Firmware	100
Motor PWM ($u_1, u_2, u_3, u_4 \in [0, 65535]$)	Firmware	100
Battery voltage V_{bat} [V]	Firmware	10

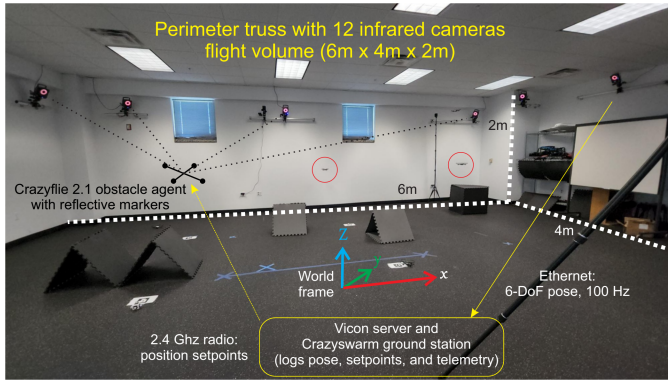


Fig. 1: Experimental setup: a Crazyflie 2.1 inside the 12-camera Vicon arena, tracked at 100 Hz over a $6 \times 4 \times 2$ m volume.

(6.4 mm diameter) in an asymmetric configuration enable unique rigid-body identification at 100 Hz with sub-millimeter accuracy.

B. Data Acquisition Pipeline

Data collection uses a custom ROS 1 framework that communicates with the Crazyflie via cflib over a Crazyradio PA USB dongle. The experimental setup is shown in Fig. 1. Three concurrent data paths operate during each flight:

Vicon path. A `vrpn_client_ros` node publishes `geometry_msgs/PoseStamped` messages on a per-rigid-

body topic at 100 Hz. The `ViconRecorder` module subscribes to this topic, computes linear velocity by first-order backward difference, converts the quaternion to Euler angles, and writes each sample to a CSV file. On every callback, the pose is also forwarded to the Crazyflie EKF through `cflib`'s `send_extpose()` to provide the onboard estimator with an external position reference.

Firmware telemetry path. The `CflibLogger` module configures firmware log blocks from a YAML specification. Each block declares a set of firmware variables, their data types, and a polling frequency, which the `cflib` fetches over the CRTP radio protocol. On reception, each sample is stamped with both the host wall-clock time and the Crazyflie firmware tick (milliseconds since boot) and written to a per-block CSV file. Table II enumerates the recorded signals and their nominal polling rates. A single Crazyradio link is bandwidth-limited, so each block is delivered below its nominal rate and is resampled onto the uniform 100 Hz Vicon grid; raw per-block sample counts are released so the interpolated fraction of any signal is recoverable. Files carry both the Vicon quaternion and Euler angles in a fixed world frame.

Command path. The `ExperimentRunner` orchestrates the flight sequence. High-level commands (`takeoff`, `land`) use the firmware's onboard trajectory planner, which generates smooth polynomial setpoints internally. During trajectory execution, position setpoints are streamed at 100 Hz via `send_position_setpoint()`, which bypasses the high-level planner and feeds the position controller directly.

C. Time Synchronization

Three clocks run independently during each flight: the Vicon Tracker, the Crazyflie firmware tick counter, and the ROS host. The host timestamps Vicon poses and firmware packets on arrival, which places both streams in a shared reference frame, but a residual offset remains from radio transmission latency (typically 2 ms to 5 ms) and operating-system scheduling jitter.

We estimate this offset by cross-correlating the onboard gyroscope rate $\omega_{\text{gyro}}(t)$ with the angular velocity $\omega_{\text{vicon}}(t)$

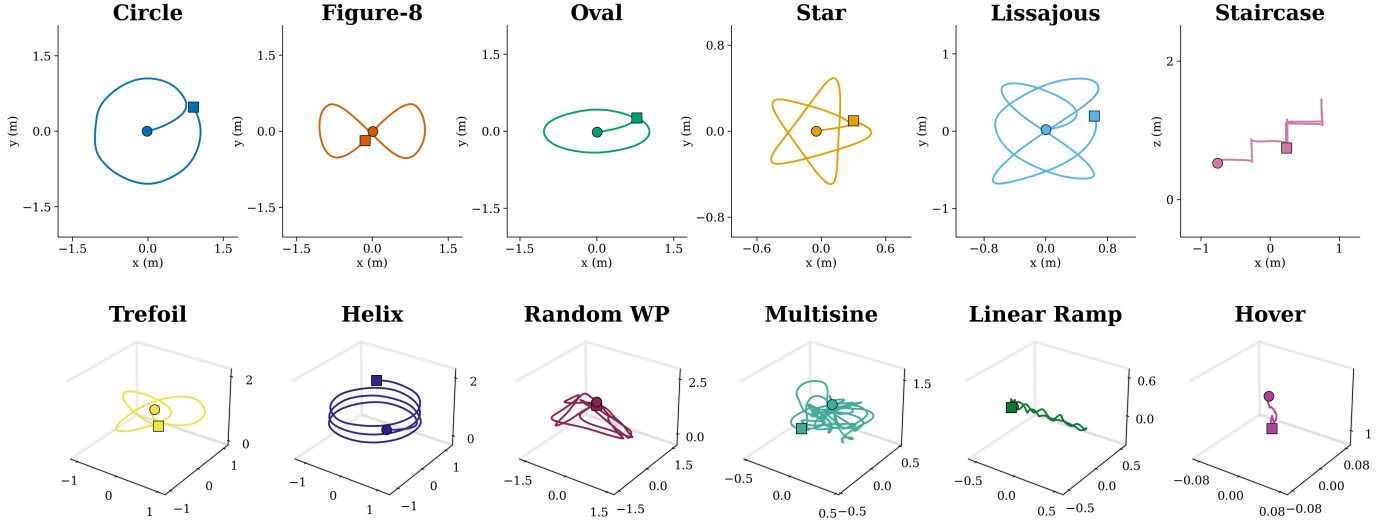


Fig. 2: All 12 NanoBench trajectory types visualized from recorded Vicon ground-truth data. Categories A (excitation), B (geometric tracking), and C (battery-drain hover).

TABLE III: Category B: tracking trajectory parameters.

Trajectory Type	Complexity Mode	Scale (m)	Speed (m/s)
Circle	Continuous yaw	$r = 1.0$	0.5, 0.75, 1.0
Figure-8	Direction reversal	$A \in \{1.0, 1.3\}$	0.5, 0.75, 1.0
Oval	Asymmetric axes	1.0×0.4	0.5, 0.75, 1.0
Star	Sharp transients	$r \in \{0.6, 1.0\}$	0.5, 0.75, 1.0
Lissajous (3:2)	Dense planar	$A = 0.6$	0.5, 0.75, 1.0
Trefoil Knot	3D oscillation	$\Delta z = 0.15$	0.5, 0.75, 1.0
Helix	Sweeping altitude	$\Delta z = 1.0$	0.5, 0.75, 1.0
Linear Ramp	1D acceleration	$L = 2.0$	0.5, 0.75, 1.0
Random Waypoint	Unstructured 3D	Volume bound	0.75
Staircase	Vertical steps	$\Delta z = 0.5$	0.5

obtained by differentiating Vicon quaternions through a Savitzky–Golay filter. Both signals are interpolated onto a common 1 ms grid and mean-removed, and the offset is

$$\Delta t^* = \arg \max_{\Delta t \in [-2, 2]s} \sum_k \omega_{\text{gyro}}(t_k) \cdot \omega_{\text{vicon}}(t_k + \Delta t). \quad (1)$$

The sharp rotational features present during maneuvers localize the correlation peak to the 1 ms grid resolution, and the pipeline logs the peak correlation for each flight as an internal alignment-quality check. Once Δt^* is found, firmware signals are shifted to the Vicon time base and resampled onto a uniform 100 Hz grid by linear interpolation. Each recording is then written to a single CSV file with all modalities on one time axis.

D. Trajectory Design

The trajectory taxonomy comprises three categories and 12 trajectory types designed to jointly excite the platform’s dynamic modes: system identification excitation, geometric tracking at multiple speeds, and long-duration battery-drain hover. Fig. 2 shows all 12 types from recorded flight data.

Category A: system identification excitation. Multi-sine trajectories superimpose sinusoidal position commands at logarithmically spaced frequencies from 0.1 Hz to 5 Hz:

$$x(t) = \sum_{n=1}^{N_f} \frac{A}{n} \sin(2\pi f_n t), \quad y(t) = \sum_{n=1}^{N_f} \frac{A}{n} \cos(2\pi f_n t + \frac{\pi}{4}), \quad (2)$$

where A is the amplitude, f_n are logarithmically spaced, and the phase offset between axes ensures simultaneous two-axis excitation. Category A consists of a single trajectory type (A1b_multisine_sysid) executing a 60 s 3D multi-sine at an altitude of ≈ 1.1 m with $N_f = 15$ components in $[0.1, 5]$ Hz, amplitudes of 0.8 m (horizontal) and 0.35 m (vertical), and smooth 2 s fade-in and fade-out windows.

Category B: geometric trajectory tracking. Ten geometric trajectories exercise complementary dynamic modes at multiple speeds (Table III). Most shapes are parameterized symmetrically, e.g., circular paths $x(t) = r \cos(\omega t)$, $y(t) = r \sin(\omega t)$ with $\omega = v/r$. Execution speeds range from 0.5 ms^{-1} to 1.0 ms^{-1} , the practical operating envelope of nano-quadrotors in confined environments.

Category C: battery drain. Four long-duration hover flights (C4_battery_drain) provide continuous recordings from ≈ 4.2 V down to ≈ 3.1 V while the vehicle holds position at ≈ 1.0 m, capturing voltage-related thrust degradation across the full discharge curve. These flights last 101–134 s and support voltage-conditioned thrust modeling and low-voltage evaluation conditions.

E. Benchmark Task Formulation

NanoBench establishes standardized protocols for three evaluation tasks.

Task 1: system identification. Given the initial state and the sequence of motor commands $\{u_i^{(k)}\}$, a model must predict the 6-DoF state trajectory $\hat{x}$ over open-loop horizons of up to $H = 50$ steps (500 ms at 100 Hz); results are reported at $h \in$

$\{1, 10, 50\}$. The reference formulation is rigid-body dynamics in the world frame $\mathcal{W}$:

$$m\ddot{\mathbf{p}} = \mathbf{R}(\mathbf{q}) \begin{bmatrix} 0 \\ 0 \\ \sum_i T_i \end{bmatrix} - mg\mathbf{e}_3 - \mathbf{D}_t\dot{\mathbf{p}}, \quad (3)$$

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} - \mathbf{D}_r\boldsymbol{\omega}, \quad (4)$$

where $\mathbf{R}(\mathbf{q}) \in SO(3)$ is the body-to-world rotation, $\mathbf{D}_t$ and $\mathbf{D}_r$ are diagonal drag matrices, and $\boldsymbol{\tau}$ is the torque vector. Training uses the Category A excitation flights at nominal voltage ($V > 3.8$ V); testing uses held-out Category B tracking flights.

For each rollout initialized at $t \in \mathcal{T}$, the model generates open-loop predictions $\hat{x}_{t+h|t}$ for $h = 1, \dots, H$. The component-wise prediction error at horizon h is

$$e_{t,h}^{(c)} = \begin{cases} \|x_{t+h}^{(c)} - \hat{x}_{t+h|t}^{(c)}\|_2, & c \in \{p, v, \omega\}, \\ 2 \operatorname{atan2}(\|q_v\|_2, q_w), & c = R, \end{cases} \quad (5)$$

where $x^{(c)}$ denotes the corresponding Euclidean state component and $[q_v \ q_w]^\top = q_{t+h}^{-1} \otimes \hat{q}_{t+h|t}$ is the relative quaternion. The mean absolute error at horizon h and its cumulative H -step variant are

$$\operatorname{MAE}^{(c)}(h) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} e_{t,h}^{(c)}, \quad \operatorname{MAE}_{1:H}^{(c)} = \sum_{h=1}^H \operatorname{MAE}^{(c)}(h). \quad (6)$$

Task 2: controller benchmarking. This task evaluates closed-loop tracking of Category B reference trajectories $\mathbf{p}_{\text{ref}}(t)$. Evaluation flights are matched on initial battery voltage ($\Delta V \leq 0.05$ V) to isolate controller performance from voltage-induced thrust variation. The position tracking error at time step k is

$$e_p(k) = \|\mathbf{p}(t_k) - \mathbf{p}_{\text{ref}}(t_k)\|_2. \quad (7)$$

Aggregate performance is reported as position RMSE, average displacement error (ADE, the mean of e_p), velocity RMSE, mean heading error $\bar{e}_\psi$, and a divergence rate defined as the percentage of time steps with $e_p(k) > 0.5$ m. Real-flight controllers are evaluated over repeated flights and reported as mean $\pm$ standard deviation; offline controllers use deterministic rollouts and are reported without repetition.

Task 3: state estimation. This task isolates the accuracy of lightweight onboard estimators against Vicon ground truth. The estimator is configured at takeoff and runs for the duration of that flight. Estimator output $\hat{\mathbf{p}}(t_k)$ is evaluated against ground truth $\mathbf{p}_{\text{gt}}(t_k)$ using the absolute trajectory error (ATE) after rigid alignment by Horn's $SE(3)$ method:

$$\text{ATE} = \left(\frac{1}{N} \sum_{k=1}^N \|\mathbf{p}_{\text{gt}}(t_k) - \mathbf{T}\hat{\mathbf{p}}(t_k)\|_2^2 \right)^{1/2}, \quad (8)$$

where $\mathbf{T}$ is the rigid transformation returned by the alignment. Additional metrics are the relative trajectory error (RTE) over sliding path-length windows, per-axis velocity residuals, and attitude RMSE in Euler angles. All flights of a trajectory type

TABLE IV: Trajectory categories in the NanoBench dataset.

Category	Trajectory types	Flights	Flight time	Samples
A: Excitation	Multi-sine (3D)	4	4.6 min	27,518
B: Tracking	Circle, figure-8, oval, star, trefoil, Lissajous, helix, ramp, random waypoints, staircase	164	96.4 min	578,623
C: Battery drain	Battery-drain hover	4	6.0 min	35,774
Total		172	107.0 min	641,915

are pooled per speed regime and reported as mean $\pm$ standard deviation across runs, with the number of runs N stated per condition.

IV. EXPERIMENTAL VALIDATION AND RESULTS

A. Dataset Statistics

NanoBench contains 172 flight recordings spanning the 12 trajectory types of Categories A–C, with a total aligned flight time of 107 min and 641,915 synchronized samples at 100 Hz. Table IV reports per-category statistics.

Battery voltage across the dataset spans 3.08 V to 4.2 V, which covers the full operational discharge curve of a single-cell LiPo. Category B tracking flights operate in the mid-to-nominal voltage regime (3.8 V to 4.2 V), while Category C hovers capture the complete discharge from ≈ 4.2 V to ≈ 3.1 V.

The synchronization pipeline (Section III-C) was applied to all 172 recordings. The estimated clock offset has a median of 282 ms with a standard deviation of 17.3 ms across flights. This offset is dominated by the difference between host wall-clock initialization and the firmware tick origin; radio transmission contributes only 2 ms to 5 ms of additional jitter, which is reflected in the standard deviation. After correction, the residual alignment error is bounded by the 1 ms search grid.

B. Task 1: System Identification Baselines

1) *Baselines:* We evaluate five baseline models of increasing complexity, adapted from Busetto et al. [23]. All learned models are trained on the multi-sine excitation flights using a temporal 80/20 split, in which the first 80 % of each flight forms the training set and the final 20 % the validation set, so that no prediction window spans both. Testing uses held-out Category B tracking flights (Table III) that contribute no training data. Below, we briefly explain the baselines for Task 1.

Naive model. A constant extrapolation model that predicts $\hat{\mathbf{x}}(t_k + h) = \mathbf{x}(t_k)$ for all horizons h , which serves as a lower bound.

Physics model. A rigid-body model implementing (3)–(4) with a quadratic motor-thrust map and fourth-order Runge–Kutta integration at 100 Hz. The bare airframe weighs 27 g; with three retroreflective markers and the wireless-charging

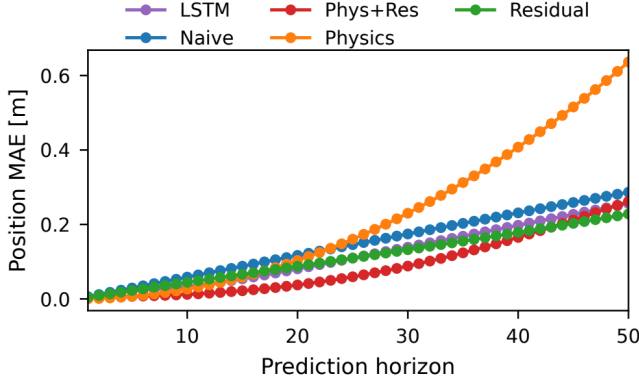


Fig. 3: Task 1: position MAE vs. prediction horizon. The physics model is most accurate at $h = 1$ but diverges beyond roughly $h = 15$ steps; the physics+residual hybrid model achieves the lowest cumulative error.

deck attached, the measured flying mass is 40.85 g, which all dynamics models use. Inertia and drag parameters are taken from [23]. Because no torque model is fitted, angular acceleration is held at zero ($\dot{\omega} \equiv 0$), so attitude is propagated from the initial angular rate. This model is not learned; it uses fixed, calibrated parameters.

Residual MLP. A feedforward network operating in the 12-D state space (position, velocity, $SO(3)$ rotation vector, angular velocity) predicts a residual $\Delta \mathbf{x}$ on top of the current state. The network consists of five fully connected layers with 64 hidden units and ReLU activations. It is trained to minimize a horizon-weighted mean-squared error over 50-step prediction windows using Adam with a cosine-annealed learning rate.

Physics + residual. A hybrid model that composes the physics baseline with the residual MLP. The physics model is rolled out to obtain a normalized prediction $\hat{\mathbf{x}}_{\text{phys}}$, and the residual network predicts a correction in the same normalized space.

LSTM. A recurrent model conditioned on the initial state $\mathbf{x}_0$ and the sequence of motor commands. A single-layer LSTM with 64 hidden units processes the control sequence and outputs per-step state increments that are accumulated into predicted trajectory. As in the MLP case, training minimizes a horizon-weighted MSE over 50-step windows.

For all learned models we use batch size 256, an exponential temporal weighting factor $\lambda = 0.1$ in the loss, and train for 500 epochs.

2) *Results and Analysis:* Table V reports MAE per state component at $h \in \{1, 10, 50\}$ steps and the cumulative score over $h = 1, \dots, 50$ steps and the cumulative score over $h = 1 \dots 50$. Based on Table V and Fig. 3, we find the following three observations:

First-principles accuracy degrades rapidly beyond short horizons. At $h=1$ (10 ms), the physics model achieves sub-millimeter position MAE (0.3 mm), significantly lower than naive extrapolation (5.7 mm), which shows that rigid-body dynamics accurately describe the immediate state evolution

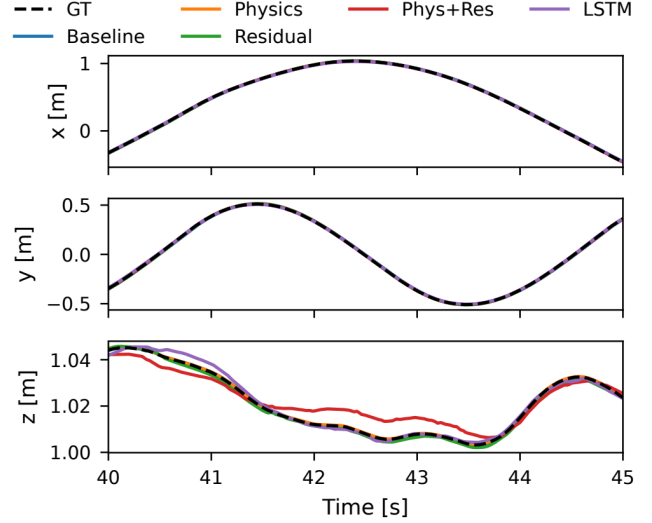


Fig. 4: Task 1: predicted (colored) versus Vicon ground-truth (black) trajectories for each baseline on a held-out Category B helix sequence, per axis.

of the platform. However, the advantage reverses sharply at longer horizons, by $h=50$, physics-model position MAE reaches 636 mm, 2.2 times worse than naive. The velocity channel reveals the mechanism: physics-model velocity MAE grows from 51 mm s^{-1} at $h=1$ to 2.57 m s^{-1} at $h=50$, which indicates that small thrust-modeling errors compound through integration and corrupt the translational state.

The hybrid architecture achieves the best cumulative accuracy. The physics + residual model attains the lowest cumulative position error (4.41 m), ahead of the residual MLP (5.68 m) and the LSTM (5.92 m). The advantage is most distinct at $h=10$, where it reaches 12.1 mm position MAE, 3.7 times lower than the residual MLP and 2.7 times lower than the LSTM. At $h=50$, however, the pure residual MLP achieves the lowest position MAE (228 mm), which suggests that the physics component eventually introduces compounding errors that the residual correction only partially absorbs. The crossover between these two models near $h = 30$ defines a practical horizon boundary for each architecture.

Translational and rotational objectives trade off under a joint loss. The LSTM achieves competitive velocity MAE (0.48 m s^{-1} at $h=50$, second only to naive), but its angular-velocity MAE reaches 5.15 rad s^{-1} , an order of magnitude above the naive baseline (0.45 rad s^{-1}). The physics model produces identical angular velocity predictions to naive at all horizons because the implementation holds $\dot{\omega} \equiv 0$ (Section IV-B1), which reflects the difficulty of modeling torque transients from coreless DC motors without dedicated angular acceleration data. Fig. 4 visualizes per-axis predictions against ground truth on a held-out helix as a test sequence.

TABLE V: Task 1: MAE per state component at horizons $h = 1, 10, 50$ steps. The italic column reports the cumulative error (sum of MAEs over $h = 1, \dots, 50$). Best value per column in bold.

Model	MAE $_{p,h}$ [m]				MAE $_{v,h}$ [m/s]				MAE $_{R,h}$ [rad]				MAE $_{\omega,h}$ [rad/s]			
	$h=1$	$h=10$	$h=50$	$h=1:50$	$h=1$	$h=10$	$h=50$	$h=1:50$	$h=1$	$h=10$	$h=50$	$h=1:50$	$h=1$	$h=10$	$h=50$	$h=1:50$
Naive	0.0057	0.0585	0.2866	7.3866	0.0085	0.0786	0.3542	9.4178	0.0033	0.0285	0.0915	2.7563	0.0402	0.2602	0.4524	17.1131
Physics	0.0003	0.0265	0.6364	10.9641	0.0511	0.5085	2.5673	65.0322	0.0019	0.0198	0.1425	3.2693	0.0402	0.2602	0.4524	17.1131
Residual	0.0045	0.0445	0.2282	5.6810	0.0121	0.1096	0.6708	16.9002	0.0084	0.1051	0.1971	8.8737	0.2043	1.1173	1.3046	54.1783
Phys+Res	0.0036	0.0121	0.2621	4.4071	0.0201	0.1370	1.0771	26.7469	0.0065	0.1359	0.1825	9.8645	0.8784	1.5799	1.1781	55.3050
LSTM	0.0058	0.0332	0.2562	5.9194	0.0199	0.1077	0.4782	11.9639	0.0162	0.1339	0.3059	9.8302	0.1256	0.8159	5.1525	118.0662

TABLE VI: Task 2: controller benchmarking results. Real-flight rows report mean $\pm$ standard deviation over repeated flights; offline rows are deterministic rollouts on the learned plant and carry no spread.

Type	Method	RMSE $_{p\downarrow}$ (m)	ADE $_{p\downarrow}$ (m)	RMSE $_{v\downarrow}$ (m/s)	$\bar{e}_{\psi\downarrow}$ (deg)	Div. (%) $\downarrow$
Real flight	PID	0.29 $\pm$ 0.03	0.27 $\pm$ 0.02	0.72 $\pm$ 0.14	5.82 $\pm$ 0.58	4.20 $\pm$ 2.46
	Mellinger	0.28 $\pm$ 0.00	0.27 $\pm$ 0.00	0.89 $\pm$ 0.18	5.27 $\pm$ 1.61	0.01 $\pm$ 0.01
Offline learned	BC-MLP	0.15	0.13	0.19	1.07	0.00
	BC-LSTM	0.18	0.16	0.22	0.97	0.00
	MPPI	1.32	1.13	0.72	4.50	75.22

C. Task 2: Controller Benchmarking

1) *Baselines*: We benchmark five controllers under two evaluation paradigms that separate physical hardware performance from data-driven rollout. **Real-flight** controllers execute onboard the platform with Vicon-aided state estimation: (1) **PID**, the firmware’s default cascaded PID controller; and (2) **Mellinger**, a geometric tracking controller on $SE(3)$ [30]. **Offline-learned** controllers are trained from logged flight data and evaluated by closed-loop rollouts against a learned dynamics model: (3) **BC-MLP** and (4) **BC-LSTM**, behavior-cloning policies trained by supervised imitation of the onboard PID using the imitation framework [33]; and (5) **MPPI** [34], a model predictive path integral controller using a neural forward model trained on the same data, run with a 10-step (100 ms) horizon, 128 sampled rollouts, temperature $\lambda = 0.1$, and exploration noise $\sigma = 0.02$ on normalized commands. Because the physical environment and the learned simulator differ fundamentally, hardware and offline metrics are reported separately in Table VI and are not directly comparable.

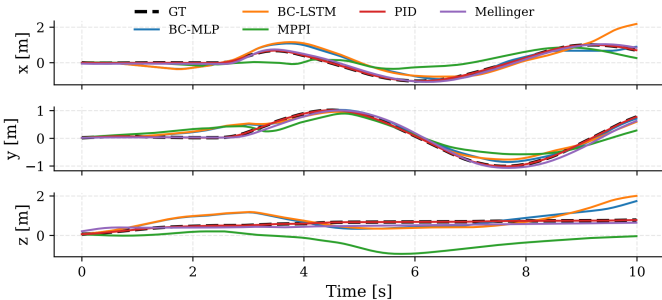


Fig. 5: Task 2: closed-loop position (m) per axis for each baseline. The dashed trace is the commanded reference trajectory, not a measured vehicle state. Real-flight (PID, Mellinger) and offline (BC-MLP, BC-LSTM, MPPI) trajectories.

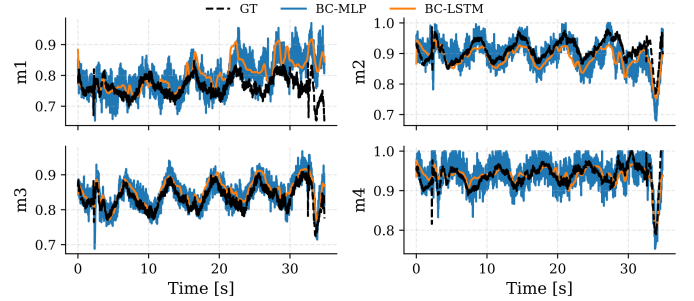


Fig. 6: Task 2: normalized motor PWM commands over time for the two behavior-cloning policies against the logged expert commands (dashed). The real-flight controllers are omitted, as their commands are not generated on the learned plant.

2) *Results and Analysis*: **On hardware, the geometric controller has fewer large excursions at the cost of higher velocity error.** PID and Mellinger demonstrate nearly identical position tracking, yet they diverge significantly in robustness and velocity regulation. The cascaded PID exhibits a 4.20 % (± 2.46 %) trajectory divergence rate, whereas the Mellinger controller reduces this to 0.01 %, representing a significant improvement. This enhanced reliability stems from the Mellinger controller’s $SE(3)$ formulation, which provides broader stability guarantees that prevent the aggressive attitude divergence occasionally observed with the PID. However, the Mellinger controller yields a velocity RMSE (0.89 m s^{-1}) that is $1.24\times$ higher than that of the PID (0.72 m s^{-1}). This trade-off is consistent with the geometric controller executing sharper attitude corrections, which inherently induce velocity transients on a nano-scale platform strictly bottlenecked by its 0.6 N peak thrust authority. Heading errors remain comparable across both PID and Mellinger. Qualitative tracking performance and corresponding motor commands are visualized in Fig. 5 and Fig. 6, respectively.

Offline-learned controllers are evaluated in simulation and cannot be compared directly with real-flight. BC-MLP and BC-LSTM achieve position RMSEs of 0.15 m and 0.18 m respectively, with zero divergence, but these values reflect the idealized learned rollout plant rather than physical flight readiness; the zero-variance confidence intervals across all offline metrics confirm purely deterministic evaluation. MPPI diverges on 75.22 % of times (position RMSE 1.32 m), indicating that its sampling-based optimization is poorly matched

TABLE VII: Task 3: onboard EKF accuracy on trefoil trajectories, mean $\pm$ standard deviation across all pooled runs (N per speed regime).

Speed	N	ATE RMSE (mm)	ATE Mean (mm)	RTE 1m (mm)	Vel RMSE (m/s)	Att RMSE (deg)
Slow	11	21.1 $\pm$ 3.3	16.5 $\pm$ 3.2	28.2 $\pm$ 6.8	0.069 $\pm$ 0.015	2.12 $\pm$ 0.05
Medium	10	21.8 $\pm$ 3.0	17.2 $\pm$ 1.7	30.3 $\pm$ 2.9	0.067 $\pm$ 0.013	2.71 $\pm$ 1.43
Fast	7	3142.7 $\pm$ 3936.6	2774.4 $\pm$ 3487.9	384.2 $\pm$ 400.8	7.629 $\pm$ 8.992	29.67 $\pm$ 28.52

to the narrow actuation limits of sub-50 g platforms without domain-specific tuning.

MPPI exhibits severe performance degradation within the nano-scale operational envelope. MPPI yields a position RMSE of 1.32 m and a 75.22 % divergence rate. This indicates that the receding-horizon optimizer exploits inaccuracies of the learned forward model and that its horizon and sampling parameters require platform-specific tuning for the narrow actuation limits of sub-50 g vehicles. We report this configuration as a reference point rather than as a limit of sampling-based MPC on this platform.

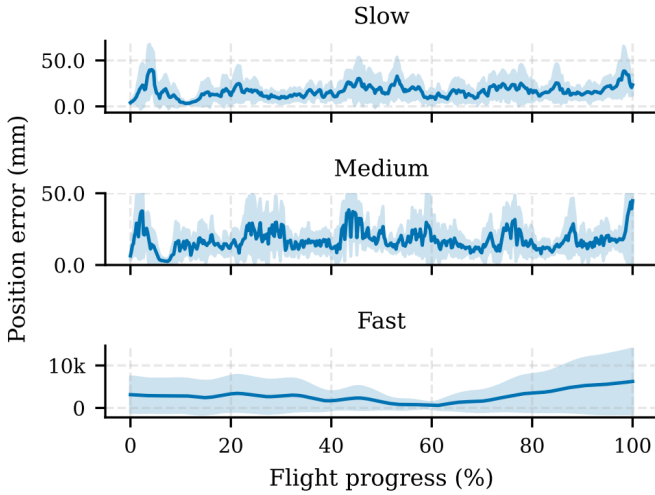


Fig. 7: Task 3: EKF position error (mean $\pm$ std, all runs pooled by speed) on trefoil trajectories. Error remains bounded at slow and medium speeds but diverges in the fast regime.

D. Task 3: State Estimation

1) *Baselines:* The **onboard EKF** runs at 100 Hz on the STM32F405 and fuses IMU data with the Vicon pose relayed through `send_extpose()` to produce the 9-DoF state estimate used in flight. Because the EKF receives the Vicon pose as an external measurement, this task characterizes the consistency of the onboard fusion (its sensitivity to external-pose latency, packet loss, and process-model mismatch) rather than dead-reckoning accuracy. Runs from both real-flight controllers of Task 2 are pooled per speed regime. Metrics are ATE after Horn’s $SE(3)$ alignment (8), RTE over 1 m path-length windows, per-axis velocity residuals, and attitude RMSE in Euler angles. Because the estimator already operates in the Vicon frame, the recovered alignment is close to the identity, so the reported ATE is effectively a direct error in the

common reference frame rather than an alignment-corrected one.

2) *Results and Analysis:* Table VII reports EKF estimation accuracy on trefoil knot trajectories over three speed regimes, with all runs pooled (mean $\pm$ std across 28 runs). At slow and medium speeds, the onboard EKF maintains position tracking within 22 mm ATE RMSE (11 and 10 runs respectively), with attitude RMSE below 3° and velocity RMSE below 0.07 m s^{-1} . At the fast regime (1.0 m s^{-1}), the EKF exhibits divergence (ATE $> 3 \text{ m}$ across 7 runs). The most plausible cause is loss or delay of external-pose packets at high angular rates on the shared radio link, from which the fixed-gain EKF does not recover within the flight; separating this effect from process-model mismatch is an open question that the released IMU, estimator, and ground-truth streams make possible to study. Fig. 7 shows the pooled position error over flight progress.

The three tasks are linked by the same platform and synchronized data: open-loop dynamics error (Task 1), closed-loop tracking (Task 2), and estimator accuracy (Task 3) all reflect the same hardware and trajectory set. NanoBench thus enables systematic comparison of methods across the autonomy stack on a platform for which no public benchmark previously existed.

E. Limitations

Three limitations bound the present scope. First, synchronization estimates a single constant offset per flight; clock skew and drift are not modelled, the residual is bounded only by the 1 ms search grid rather than measured independently, and alignment quality degrades on trajectories with little rotational excitation, so the residual should not be assumed uniform across the dataset. Second, Task 3 supplies the Vicon pose to the estimator through `send_extpose()` and evaluates against the same signal, which makes it a benchmark of Vicon-aided fusion consistency rather than of standalone onboard estimation; a dead-reckoning protocol requires re-running estimators offline from the released IMU stream. Third, Task 2’s hardware and offline rows are distinct experiments on distinct plants, and the MPPI configuration is untuned, so neither supports a general claim about predictive control at this scale. All three are addressable with the released data and code, and we leave them as future work.

V. CONCLUSION

NanoBench introduces the first open-source, multi-task benchmark for nano-scale quadrotors, comprising 172 flight recordings on the commercially standard Crazyfly 2.1 in a Vicon motion capture arena. It is distinguished from prior aerial datasets by jointly providing synchronized motor PWM commands, onboard EKF and PID controller internals, and millimeter-accurate ground truth, signals that no existing benchmark exposes at the nano scale.

Baseline evaluations across three tasks expose platform-specific behaviors that larger-vehicle benchmarks cannot replicate. In Task 1, rigid-body dynamics achieve sub-millimeter

one-step accuracy but diverge beyond a 100 ms horizon because of compounding thrust-model error, and a physics-plus-residual hybrid gives the lowest cumulative error. In Task 2, the default PID and Mellinger controllers reach similar tracking RMSE, but the geometric controller exhibits far fewer large excursions, while an untuned MPPI baseline fails on the learned plant. In Task 3, the onboard EKF maintains sub-22 mm ATE at slow and medium speeds but loses consistency in part of the fast-speed flights, a failure mode the released multi-stream logs allow future work to diagnose.

REFERENCES

- [1] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Mueller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, pp. 982–987, 08 2023.
- [2] M. O'Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural-fly enables rapid learning for agile flight in strong winds," *Science Robotics*, 2022.
- [3] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, "Data-driven mpc for quadrotors," *IEEE Robotics and Automation Letters*, 2021.
- [4] S. I. Ullah and J. Baca, "Aerocast: Probabilistic 3d trajectory prediction for non-cooperative aerial obstacles via transformer-mdn architecture," 2026. [Online]. Available: <https://arxiv.org/abs/2606.25122>
- [5] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, 2022.
- [6] L. Pichierri, A. Testa, and G. Notarstefano, "Crazychoir: Flying swarms of crazyflie quadrotors in ros," *IEEE Robotics and Automation Letters*, 2023.
- [7] J. A. Preiss*, W. Hönig*, G. S. Sukhatme, and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017.
- [8] S. Izzat Ullah, J. Baca, P. Rangel, T. Chu, and C. Rubio-Medrano, "Pof-mader: Trajectory planner in dynamic environments with improved collision avoidance," *IEEE Access*, 2025.
- [9] S. I. Ullah and A. Muhammad, "Autonomous navigation and mapping of water channels in a simulated environment using micro-aerial vehicles," in *2023 International Conference on Robotics and Automation in Industry (ICRAI)*, 2023.
- [10] R. W. Deters, G. K. Ananda Krishnan, and M. S. Selig, "Reynolds number effects on the performance of small-scale propellers," in *32nd AIAA applied aerodynamics conference*, 2014, p. 2151.
- [11] J. Förster, "System identification of the Crazyflie 2.0 nano quadcopter," Master's thesis, ETH Zurich, 2015.
- [12] G. Shi, X. Shi, M. O'Connell, R. Yu, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural lander: Stable drone landing control using learned dynamics," in *IEEE ICRA*, 2019.
- [13] D. Palossi, "On the autonomous navigation of nano-uavs," Ph.D. dissertation, ETH Zurich, 2019.
- [14] S. I. Ullah, "Uncertainty-aware probabilistic forecasting of non-cooperative dynamic obstacles for safe autonomous aerial navigation," Ph.D. dissertation, Texas A&M University-Corpus Christi, Corpus Christi, Texas, 2026.
- [15] L. Bauersfeld, E. Kaufmann, P. Foehn, S. Sun, and D. Scaramuzza, "Neurohem: Hybrid aerodynamic quadrotor model," *arXiv*, 2021.
- [16] Y. Song, A. Romero, M. Muller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, 2023.
- [17] J. Eschmann, D. Albani, and G. Loianno, "Learning to fly in seconds," *IEEE Robotics and Automation Letters*, 2024.
- [18] N. El-Sheimy and A. Youssef, "Inertial sensors technologies for navigation applications: State of the art," *Satellite navigation*, 2020.
- [19] S. I. Ullah and J. Baca, "SynTraG: A synthetic trajectory generator for non-cooperative dynamic obstacles in UAV navigation," in *Advances in Automation and Robotics Research*. Springer, 2026.
- [20] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The euroc micro aerial vehicle datasets," *The International Journal of Robotics Research*, 2016.
- [21] A. Antonini, W. Guerra, V. Murali, T. Sayre-McCord, and S. Karaman, "The blackbird dataset: A large-scale dataset for uav perception in aggressive flight," in *Int. Sym. on Exp. Robotics (ISER)*, 2018.
- [22] J. Delmerico, T. Cieslewski, H. Rebecq, M. Faessler, and D. Scaramuzza, "Are we ready for autonomous drone racing? the uzh-fpv drone racing dataset," in *IEEE ICRA*, 2019, pp. 6713–6719.
- [23] R. Busetto, E. Cereda, M. Forgiione, G. Maroni, D. Piga, and D. Palossi, "Nonlinear system identification for a nano-drone benchmark," *Control Engineering Practice*, vol. 172, p. 106871, 2026.
- [24] T.-M. Nguyen, S. Yuan, M. Cao, Y. Lyu, T. H. Nguyen, and L. Xie, "Ntu viral: A visual-inertial-ranging-lidar dataset, from an aerial vehicle viewpoint," *The International Journal of Robotics Research*, 2022.
- [25] C. Brommer, A. Fornasier, M. Scheiber, J. Delaune, R. Brockers, J. Steinbrener, and S. Weiss, "The insane dataset: Large number of sensors for challenging uav flights in mars analog, outdoor, and out-/indoor transition scenarios," *Int. J. Rob. Res.*, vol. 43, no. 8, 2024.
- [26] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld, T. Laengle, G. Cioffi, Y. Song, A. Loquercio, and D. Scaramuzza, "Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight," *Science Robotics*, 2022.
- [27] M. Wang, J. Zhou, X. Duan, D. Zhao, P. Cai, J. Zhai, X. Liu, and C. Ren, "Nonlinear system identification for quadrotors with neural ordinary differential equations," in *IEEE ICUS*, 2023.
- [28] K. Y. Chee, T. Z. Jiahao, and M. A. Hsieh, "Knode-mpc: A knowledge-based data-driven predictive control framework for aerial robots," *IEEE Robotics and Automation Letters*, 2022.
- [29] C. Llanes, Z. Kakish, K. Williams, and S. Coogan, "Crazysim: A software-in-the-loop simulator for the crazyflie nano quadrotor," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [30] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *IEEE ICRA*, 2011.
- [31] J. Baca, S. I. Ullah, and P. Rangel, "Coaxial modular aerial system and the reconfiguration applications," in *IEEE ICRA*, 2023.
- [32] D. Palossi, N. Zimmerman, A. Burrello, F. Conti, H. Muller, L. M. Gambardella, L. Benini, A. Giusti, and J. Guzzi, "Fully onboard ai-powered human-drone pose estimation on ultralow-power autonomous flying nano-uavs," *IEEE Internet of Things Journal*, 2022.
- [33] A. Gleave, M. Taueeque, J. Rocamonde, E. Jenner, S. H. Wang, S. Toyer, M. Ernestus, N. Belrose, S. Emmons, and S. Russell, "imitation: Clean imitation learning implementations," 2022.
- [34] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Reh, B. Boots, and E. A. Theodorou, "Information theoretic mpc for model-based reinforcement learning," in *ICRA*, 2017.